

Programa de Pós-Graduação em Computação Aplicada
Universidade de Brasília

Geração Automatizada de Testes Baseada em Model Checking: Um Estudo Aplicado ao e-Título Versão para Qualificação

Thiago Viana Fernandes
thiagu.vf@gmail.com

25 de fevereiro de 2026

- ▶ APIs REST são base de sistemas modernos e críticos.
- ▶ Erros de integração frequentemente não são detectados por testes tradicionais.
- ▶ Testes manuais e exploratórios possuem baixa sistematização.
- ▶ Lacuna entre modelos formais e práticas de teste de software.

- ▶ Aplicativo oficial da Justiça Eleitoral brasileira, com mais de 57 milhões de eleitores ativos.
- ▶ Arquitetura baseada em APIs REST, suportando processos críticos de identificação e emissão de documentos digitais.
- ▶ Alta complexidade nos fluxos de autenticação, combinando componentes síncronos e assíncronos.
- ▶ Demanda rigorosos requisitos de segurança, confiabilidade e disponibilidade.

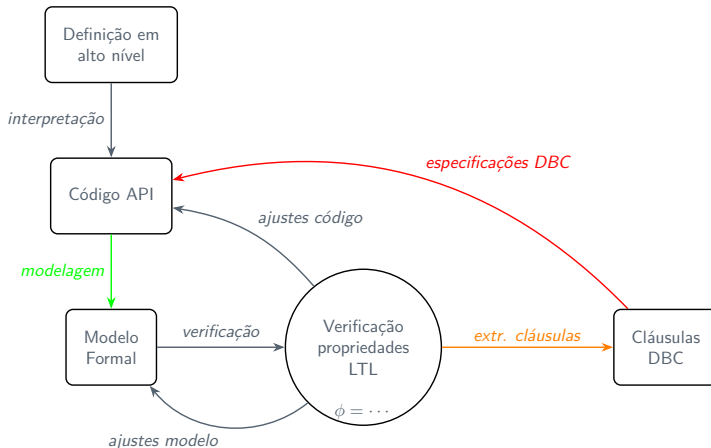
- ▶ Testes tradicionais operam apenas na implementação e não exploram exaustivamente interações emergentes.
- ▶ A eficácia do *Model Checking* esbarra no distanciamento entre o modelo formal e a execução prática no ambiente de testes.
- ▶ **Problema central:** Faltam mecanismos que transformem cenários críticos descobertos por verificação formal em restrições diretamente executáveis no código das APIs.

- ▶ **Objetivo Geral:** Desenvolver, avaliar e validar um método que guie a definição de contratos DbC a partir da análise de cenários via *Model Checking* em sistemas críticos.
- ▶ **Objetivos Específicos:**
 1. Investigar as limitações de técnicas automatizadas de teste.
 2. Explorar a viabilidade do *Model Checking* como técnica complementar.
 3. Definir método para geração de contratos DbC baseados na exploração sistemática de estados.
 4. Aplicar a metodologia no estudo de caso do e-Título.

- ▶ O cenário atual foca vastamente em estratégias de caixa-preta e, mais recentemente, IA/LLMs para automação de testes REST.
- ▶ Abordagens que empregam técnicas formais focam em invariantes e propriedades comportamentais, mas ainda são limitadas.
- ▶ **Oportunidade:** Beneficiar-se do rigor matemático na identificação de violações sutis, integrando os achados às práticas de desenvolvimento.

- ▶ **Model Checking (SPIN/Promela):** Exploração sistemática e exaustiva de todos os estados do sistema em busca de falhas lógicas.
- ▶ **LTL (Linear Temporal Logic):** Linguagem de especificação de propriedades desejadas, verificando o tempo de execução e a progressão dos estados.
- ▶ **Design by Contract (DbC):** Formalização de acordos entre cliente e fornecedor (Módulos/APIs) usando pré-condições, pós-condições e invariantes.

- ▶ Pesquisa aplicada, descritiva e experimental baseada no sistema e-Título.
- ▶ Método iterativo estruturado no ciclo PDCA (*Plan-Do-Check-Act*) para refinamento contínuo dos modelos.
- ▶ **O ciclo de verificação inclui:**
 1. Modelagem comportamental em linguagem de alto nível (Promela).
 2. Verificação de propriedades via SPIN.
 3. Análise de violações (*contraexemplos*) e ajustes.
 4. Extração das cláusulas DbC a partir da estrutura do modelo gerado.

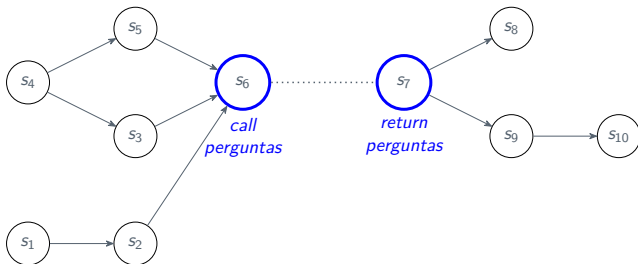


- Foco no fluxo de *onboarding*.



- Modelagem do comportamento com mapeamento explícito de interações (diagramas de estado baseados nos serviços REST reais).
- Abstração de detalhes internos (caixa-preta) para reduzir a explosão de estados, cobrindo sucesso, bloqueio ou emissões restritas.

- ▶ A extração de contratos baseia-se nas proposições atômicas presentes no modelo Promela/SPIN.
- ▶ **Rotulação:** Estados rotulados como *call* (chamada à API) e *return* (retorno da API).



- ▶ **Pré-condições:** Extraídas via interseção lógica das proposições de todos os estados imediatamente *antecessores* à chamada.



- ▶ **Pós-condições:** Interseção das proposições em todos os estados *sucessores* ao retorno do método.



- ▶ **Invariantes:** Proposições comuns não alteradas entre chamada e retorno.



- ▶ Contratos como oráculos de teste.



- ▶ As proposições lógicas identificadas pelo Model Checking são materializadas no formato OpenAPI.
- ▶ **Exemplo Prático:**
 - ▶ Proposições no modelo: $p_1 : id \neq null$, e $p_2 : questoes \in [43, 44]$.
 - ▶ Traduzido para a API: Parâmetros declarados como `required: true`, limitados usando `minimum` e `maximum`.
- ▶ Aproxima o rigor formal (garantia por construção) das verificações automatizadas de integração.

- ▶ **Adequação da Modelagem:** Representatividade do modelo Promela comparado ao sistema em produção.
- ▶ **Expressividade Contratual:** Avaliação se a cláusula DbC cobre integralmente as restrições inferidas pelo modelo.
- ▶ **Taxa de Detecção:** Medir propriedades violadas encontradas pelo modelo que fugiram à suíte atual de testes.
- ▶ **Utilidade Prática:** Avaliação qualitativa da viabilidade do método junto aos desenvolvedores no TSE.

- ▶ O uso guiado de contraexemplos gerados pelo SPIN atua como fonte rica para a engenharia de requisitos comportamentais, definindo contratos DbC mais robustos.
- ▶ **Próximas Etapas (Cronograma):**
 - ▶ Ampliar o escopo dos fluxos de *onboarding* do e-Título.
 - ▶ Realizar avaliação empírica das falhas detectadas usando os contratos gerados vs. testes tradicionais.
 - ▶ Avançar na automatização parcial da tradução de Promela/LTL para especificações DbC.
 - ▶ Submissão e defesa final da dissertação.

Obrigado!